ϕ

Calibrated Lifelong Reinforcement Learning for Efficient Adaptation to Changing Action Sets

Anonymous submission

Abstract

We study the generalization of reinforcement learning (RL) methods when the environment has an increasing number of actions. Lifelong RL provides a solution that infers the underlying structure in the action space and optimizes a policy that leverages this structure. However, existing lifelong learning works to address this issue commonly face performance degradation when generalizing to new actions, making it challenging to efficiently transfer previously learned knowledge to new actions. To tackle this issue, we first analyze the reason for the performance degradation during adaptation, and find that the degradation lies in the underestimation of the value functions when the action set changes. We then propose a calibration method for the estimation of value functions inspired by offline-to-online RL methods, which can effectively alleviate the above issue and lead to an efficient adaptation when generalizing to new actions. We further derive the regret bound of the proposed calibration method. We also design a replay buffer to store the transitions collected during learning to improve sample efficiency. Finally, we demonstrate the efficacy and efficiency of the proposed method in four tasks of the RL benchmark environment.

Introduction

Reinforcement learning (RL) is a branch of artificial intelligence that explores computational models of learning through interactions with an environment and numerical feedback (Sutton and Barto 2018). RL methods have shown remarkable success not only in gameplay (Silver et al. 2016; Mnih et al. 2015; Vinyals et al. 2019) but also in real-world applications including robotics (Ju et al. 2022), chemical synthesis (Segler, Preuss, and Waller 2018), and personalized recommendations (Afsar, Crump, and Far 2022). However, one major drawback of most RL methods is that they face challenges when the environment has an incremental action set since RL methods are mostly designed to address a mathematical formulation of decision problems known as Markov Decision Processes (MDP), in which the set of actions is fixed. In contrast, humans can choose from a new set of actions by observing them, inferring their characteristics, and making decisions that meet the tasks requirements.

Consequently, recent RL research has concentrated on algorithms that allow agents to generalize to new actions, utilizing knowledge gained from the past to speed up learning to new actions (Chandak et al. 2020; Jain, Szot, and Lim 2020; Jain et al. 2022a; Li et al. 2022). A common approach is to learn an action representation function that maps the actions into a representation space based on their environmental feedback (e.g., observations after executing an action) with unsupervised learning (Jain, Szot, and Lim 2020; Jain et al. 2022a; Li et al. 2022), and then learns a transferable policy that maps the action representations to the available actions. However, when the environment has a continuous influx of new actions, those methods fail to generalize the learned policy to the new actions continuously.

Some recent works address the above issues by employing lifelong learning to the changing action set. Lifelong RL methods infer the action representations from the environmental feedback, learn the policy for action representations, and then map them to real actions. When new actions are added to the environment, lifelong RL methods reuse the representation policy and adapt the action representations to the new actions. However, existing lifelong learning works commonly face a significant issue: when generalizing to new actions in the adaptation phase, the algorithms often show a catastrophic performance degradation during learning, i.e, the averaged total reward obtained for each episode drops in a relatively low level in a few episodes, making it challenging to rapidly transfer previously learned policy to new actions. Especially in the early stages of adaptation, existing methods often tend to ”unlearn” the policy and require a large number of samples to restore performance (Chandak et al. 2020), which is inefficient in learning.

In this paper, we take a first look at the reason for the catastrophic performance degradation, and find that the degradation lies in the underestimation of the value functions when the action space changes. We then propose a calibration method for the value functions inspired by the works on offline-to-online RL (Nakamoto et al. 2024; Kumar et al. 2020). With calibration, the estimated value functions are bounded to avoid underestimation, leading to an efficient adaptation when the action space changes, as illustrated in Figure 1. We further derive the regret bound of the proposed calibration method. In addition, we design a replay buffer to store all the transitions collected for learning to further improve the sample efficiency. To evaluate the proposed method empirically, we conduct experiments on 4 tasks from the well-known RL benchmark environment. Our experimental results show the proposed methods can signif-

icantly improve the performance and efficiency in tasks of lifelong learning with changing action sets. To be specific, our main contributions are as follows:

- We analyze the reason for performance degradation of the existing lifelong RL methods for changing action sets. To the best of our knowledge, this is the first work to address this issue.
- We propose a calibration method for value estimation in lifelong RL, which improves learning efficiency while adapting to new actions.
- We derive the regret bound theoretically on the proposed calibration method.

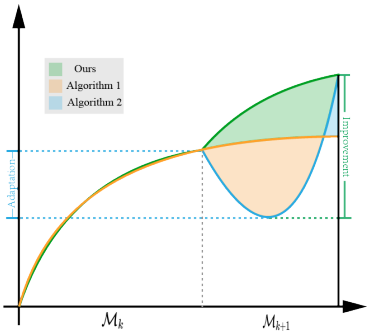


Figure 1: We study lifelong reinforcement learning with changing action spaces. When the action space changes, the existing lifelong RL method (Algorithm 2) always faces performance degradation. Meanwhile, even though most of the existing RL methods cannot adapt to changes in the environment, when the action space changes, the performance of those methods will remain stable. We develop an approach to improve the adaptation efficiency of lifelong RL methods inspired by offline-to-online methods.

Preliminaries and Notions

Reinforcement learning (Sutton and Barto 2018) is formulated as Markov decision processes (MDP): $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, R, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ is the action space, $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the state transition function, $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function and γ defines the discount factor. The agent's behavior is described by a policy function $\pi : \mathcal{S} \times \mathcal{A} \to [0, 1]$. Each episode in the MDP denotes an interaction process. For an episode with a duration of T, the cumulative reward of the t-th step is: $G_t = \sum_{i=t}^{T}(\gamma^{i-t}R(s_t, a_t))$. The goal of RL is to find a policy $\pi : S \to A$ that maximizes the cumulative reward for the whole interactive process:

$$\pi = \arg\max_{a \in A} \mathbb{E}[G_t] \tag{1}$$

Lifelong RL generalizes conventional RL to tackle a series of different tasks continuously (Taiga et al. 2023). It is mathematically formulated as Lifelong Markov decision processes (L-MDP), which is defined as $\mathcal{L} = (\mathcal{M}_0, \mathcal{E}, D)$. $\mathcal{M}_0 = (\mathcal{S}, \mathcal{A}, \mathcal{P}, R, \gamma, d_0)$ is a base MDP and d_0 is the initial distribution of D. When the available set of actions changes, the agent observes a new set of discrete actions $\mathcal{A}_k$, transforming $\mathcal{M}_{k-1}$ into $\mathcal{M}_k$, where k is the number of times the action set is changed (Chandak et al. 2020).

The lifelong RL agent aims to find a policy π_k that can maximize the expected cumulative discount rewards. The state value function of π_k is:

$$v^{\pi_k}(s) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t R_t | s, \pi_k]$$

In order to adapt to the changing action set, lifelong RL methods learn an action representation function and infer the underlying structure of actions with this function (Mazoure et al. 2022; Raileanu et al. 2020). The policy is composed of two parts: the action representation policy $\beta : \mathcal{S} \times \mathcal{E} \to [0, 1]$ which maps the states to a probability distribution over action representations, and a mapping function $\phi : \mathcal{E} \times \mathcal{A} \to [0, 1]$ that maps the action representations to a probability over real actions. Denoting e_t as the latent action representation at time t, the following equation describes its relationship with the action space.

$$e_t \sim \beta(\cdot \mid S_t), \quad A_t = \phi(e_t) \tag{2}$$

Denoting π_θ as the parameterized policy which maps the states to the probability of real actions, then the following equation denotes the relationship between β_{π_θ} and π_θ:

$$\pi_\theta(a|s) := \int_{\phi_{\pi_\theta}^{-1}(a)} \beta_{\pi_\theta}(e|s)\, \mathrm{d}e \tag{3}$$

With action representations, the action value functions are estimated under the action representation policy β:

$$Q^{\pi_\theta}(s, a) = Q^{\beta_{\pi_\theta}}(s, e) \tag{4}$$

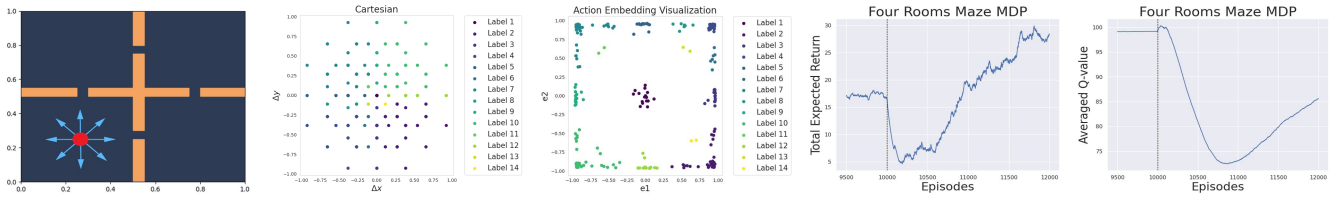
The value function with action representations can be defined as:

$$v^{\pi_\theta}(s) = \sum_{a \in \mathcal{A}} \int_{\phi_{\pi_\theta}^{-1}(a)} \beta_{\pi_\theta}(e|s) Q^{\pi_\theta}(s, a) \mathrm{d}e \tag{5}$$

When the action set changes, lifelong RL typically involves two phases: first, collects samples and offline learns the action representation function $\hat{\phi}$ to adapt to the changed environment. Second, learn the action representation policy β w.r.t. $\hat{\phi}$ with online interaction of the environment. Note that $\hat{\phi}$ is a parameterized function of ϕ.

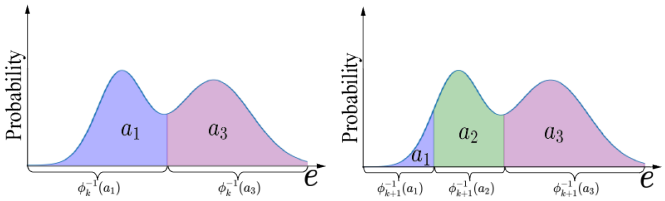
The Performance Degradation in Lifelong RL when facing a Changing Action Space

Existing lifelong RL methods for changing action sets reuse the previously learned knowledge and then adapt the past knowledge by fine-tuning the learned functions with the samples collected from the changed environment when the action space changed (Chandak et al. 2020; Jain, Szot, and Lim 2020). However, we empirically find that the past learned knowledge, especially the value functions for action representations, may mislead the fine-tuning process during policy improvement. We start by analyzing the reason for the performance degradation of LAICA during the adaptation of new actions.



(a) Four Rooms Maze (b) 2D Action Displacements (c) Action embeddings (d) Total Expected Return (e) Averaged Q-Value

Figure 2: (a) The Four Rooms maze environment (b) The displacement of each action. (c) The learned 2D action representation vectors for the 256 available actions, which are clustered using DBSCAN. The colors of actions are corresponding to the action representations in (b). (d) The performance of LAICA is measured in the total return for each episode. (e) The averaged Q-values of LAICA during learning.



(a) Probability Distribution of actions in $\mathcal{M}_k$ (b) Probability Distribution of actions in $\mathcal{M}_{k+1}$

Figure 3: Illustration of the probability induced for three actions by the probability density of $\beta(e|s)$ on a 1-D embedding space. The x-axis represents the embedding and the y-axis represents the probability. The colored regions represent the mapping $a = \phi(e)$, where each color is associated with a specific action. When the environment MDP changes from $\mathcal{M}_k$ to $\mathcal{M}_{k+1}$, Some action representations are no longer mapped to the previous actions (i.e., a_1 in the left) but are mapped to some completely new actions (i.e., a_2 in the right).

Empirical Analysis

We evaluate the LAICA algorithm on the Four Rooms maze task from MiniGrid [1], where the agent controls a robot with eight actuators in different directions 2a. The agent's goal is to reach the goal by moving the 8 actuators. Therefore, there are 2^8 possible actions in total. In the beginning, the agent only knows part of the actuators, and the actuators will continually activate during the learning process, thus simulating the lifelong RL scenario. The details of the Four Rooms maze task can be found in the appendices. We first analyze the learned action representation of LAICA in the Four Rooms Environment. We report two results of the learned action representations: action embedding vectors (Figure 2c) and the displacement of the agent after executing an action (Figure 2b). The results show that similar action embeddings lead to similar actions. However, their real Q-values for some specific states may differ significantly, as the displacements they induce are different.

[1] https://minigrid.farama.org/environments/minigrid/FourRoomsEnv/

We also analyze the averaged Q-value during the learning process of LAICA. Figure 2d shows the normalized total return of each episode, where in episodes 10000 new actions are added into the action space, and Figure 2e shows the corresponding averaged Q-values during learning. Although LAICA can adapt to the changes in action spaces, it always faces performance degradation during the early stages of fine-tuning. Meanwhile, with the involvement of new actions, the average Q-value in each episode initially drops and then rises. The episodes during which the Q-value decreases mostly coincide with those in which the policy performance degrades. The agent tends to *foget* the learned Q-values in the prior stage, and requires a large number of interactions to recover its performance in the prior stage. We will further analyze the unlearning problem in LAICA during the adaptation to the new actions.

Why does LAICA unlearn initially?

To understand why LAICA unlearns initially during the adaptation stage, let's recall the learning process of LAICA. When the action space changes, i.e., when the underlying MDP $\mathcal{M}_k$ transitions to $\mathcal{M}_{k+1}$, and ϕ_{k+1} is initialized to ϕ_k, LAICA contains two phases of learning:

- *Phase 1*: Collect feedbacks for the environment and learns $\hat{\phi}_{k+1}$ and the inverse dynamics function ψ_{k+1} to adapt to ϕ_{k+1}. This process is carried out offline and utilizes the knowledge from $\hat{\phi}_k$ and ψ_k for adaptation.
- *Phase 2*: Perform policy improvement of the action representation policy β_{k+1} by interacting with the environment online with the learned functions $\hat{\phi}_{k+1}$ and ψ_{k+1} previously.

As the number of available actions increases, the value functions, which are estimated with action representations, remain consistent without updating during Phase 1, but the action representation function $\hat{\phi}_k$ has been updated to $\hat{\phi}_{k+1}$. Some action representations are no longer mapped to the previous actions but are mapped to some completely new actions.

Figure 3a gives an illustration of the probability density of the real actions a_1, a_3 of $\beta_k(e|s)$ on a 1-D embedding space. When the environment MDP transitions from $\mathcal{M}k$

to $\mathcal{M}k+1$ and a new action a_2 is introduced, the embedding region of the original $\phi_k^{-1}(a_1)$ is split into $\phi_{k+1}^{-1}(a_1)$ and $\phi_{k+1}^{-1}(a_2)$. Meanwhile, before policy improvement in the Phase 2, past representation policies and value functions are retained, i.e., $Q_{k+1}(\hat{\phi}_{k+1}^{-1}(a), s) := Q_k(\hat{\phi}_k^{-1}(a), s)$ remains consistent. When Phase 2 begins, the representation policy β_k is optimized through online interactions with the environment.

Considering the situation in Figure 3b, based on the learned behavior policy β_k at the prior stage, a_2 has a higher probability of being selected than a_1. At the beginning of Phase 2, the policy β_{k+1} is fine-tuned according to the old value function Q_k and experiences. Due to the insufficient samples in the early learning stages, the Q-values for a_1, i.e., $Q_{k+1}(\hat{\phi}_{k+1}^{-1}(a_1), s)$ may be overestimated or underestimated. If an overestimation occurs, then during the optimization of the policy gradient, the probability of action of a_1 will gradually increase during the learning process, thus collecting more samples to correct the overestimated Q-values in certain episodes. In contrast, if an underestimation occurs, the opposite happens: the probability of action of a_1 may gradually decrease during the learning process, and the correction of $Q_{k+1}(\hat{\phi}_{k+1}^{-1}(a_1), s)$ will need more samples with more episodes. This underestimation may be harmful to policy improvement, as it slows the policy improvement process. In fact, during the empirical analysis(Figure 2), we observed the average Q-value at the beginning of Phase 2 drops and require a certain number of episodes to recover. Therefore, we speculate that there exist underestimations of the value functions at the beginning of Phase 2, leading to the catastrophic drop in the normalized total return.

To summarize, Current lifelong RL methods suffer from performance degradation and unlearning the Q-values when the number of actions increases. We find that this phenomenon may be due to the underestimation of Q-values at the beginning of online learning. Therefore, in this paper we attempt to develop a calibration method for the Q-values during policy improvement, aiming to alleviate the underestimation problem.

Methodology

Calibration of the Value Functions

As illustrated before, the underestimation of the Q-values for action representations when new actions are introduced may cause performance degradation in lifelong RL. We observed a similar issue in offline-to-online RL, where the underestimation is also occurs and leads to performance degradation when online learning begins. Therefore, existing offline-to-online RL methods such as Cal-QL (Nakamoto et al. 2024) adopted a calibration method for value estimation. Inspired by Cal-QL, we develop a calibration method for lifelong RL for changing action space. In Cal-QL, the calibration of Q-values is defined as:

$$\mathbb{E}_{a\sim\pi}\left[Q_\theta^\pi(s,a)\right] \geq \mathbb{E}_{a\sim\mu}\left[Q_\theta^\mu(s,a)\right] \tag{6}$$

where $s \in \mathcal{S}$, μ is a reference policy.

This inequality indicates that by using the true value of the reference policy as a lower bound, Cal-QL can prevent the policy from learning excessively low-Q values. However, in lifelong RL, the latent action space $\mathcal{E}$ is continuous, and the Q-values is estimated based on the action representations, whereas in Cal-QL (Nakamoto et al. 2024), the goal is to calibrate the value estimation under discrete actions. Therefore, to address this issue, we can rewrite the above equation as the calibration suitable for action representation space due to the independence of β and ϕ:

$$\begin{aligned}\sum_{a\in\mathcal{A}}\int_{\phi_\pi^{-1}(a)}\beta_\pi(e|s)Q_\theta^{\beta_\pi}(s,e)\mathrm{d}e \geq \\ \sum_{a\in\mathcal{A}}\int_{\phi_\mu^{-1}(a)}\beta_\mu(e|s)Q_\theta^{\beta_\mu}(s,e)\mathrm{d}e\end{aligned} \tag{7}$$

By introducing the calibration, the learned Q-values is constrained to be greater than V^μ, effectively addressing the underestimation issue when new actions are involved. The following regularizer is proposed in Cal-QL to calibrate the estimated Q-values:

$$\begin{aligned}\mathcal{R}(\theta) =& \mathbb{E}_{s\sim\mathcal{D}}\left[\max\left(Q_\theta^\pi(s,a), V^\mu(s)\right)\right] \\ &- \mathbb{E}_{s,a\sim\mathcal{D}}\left[Q_\theta^\pi(s,a)\right]\end{aligned} \tag{8}$$

Since lifelong RL methods optimize the representation policy in the latent action space, we need to derive an applicable regularization method with action representations. Therefore, rewrite the definition of value functions for action representations in Equation 5, we obtain:

$$\begin{aligned}v^\pi(s) &= \sum_{a\in\mathcal{A}}\int_{\phi_\pi^{-1}(a)}\beta_\pi(e|s)\,Q^\pi(s,a)\,\mathrm{d}e \\ &= \sum_{a\in\mathcal{A}}\int_{\phi_\pi^{-1}(a)}\beta_\pi(e|s)\,Q^{\beta_\pi}(s,e)\,\mathrm{d}e := v^{\beta_\pi}(s)\end{aligned} \tag{9}$$

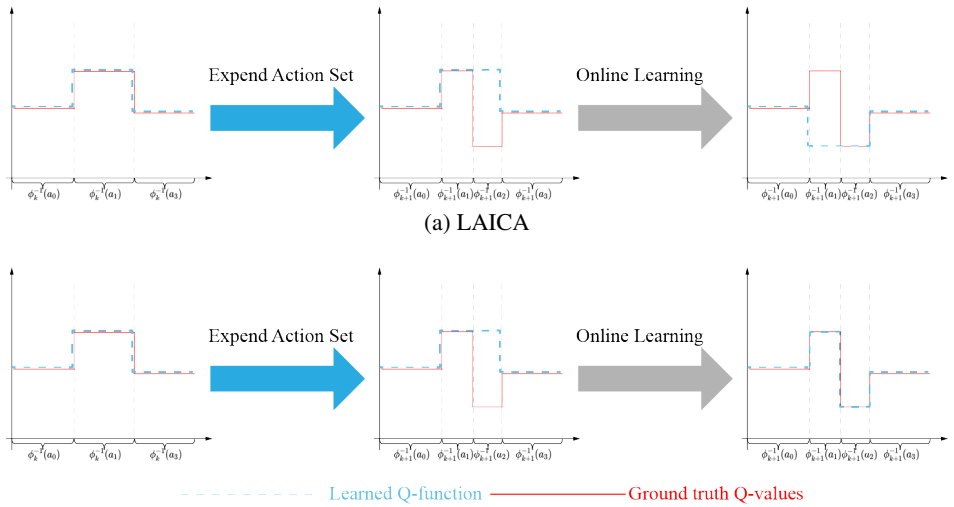
Thus, we can rewrite the regularization term in Equation 8 into a regularizer with regard to the action representation policy β in the latent action space $\mathcal{E}$:

$$\begin{aligned}\mathcal{R}(\theta) =& \mathbb{E}_{s\sim\mathcal{D}}\left[\max\left(Q_\theta^{\beta_\pi}(s,e), V^\mu(s)\right)\right] \\ &- \mathbb{E}_{s,a\sim\mathcal{D}}\left[Q_\theta^{\beta_\pi}(s,e)\right]\end{aligned} \tag{10}$$

With the above equation, we can calibrate the learned Q-values in lifelong RL to avoid underestimations during the adaptation to the new actions.

Regret Bound Analysis

To analyze the regret bound of the proposed calibration method, we introduce the following regret definition:$Reg_k(N_k) := \mathbb{E}_{s\sim\mathcal{D}}\sum_{n=1}^{N_k}\left[V^{\pi^\star}(s) - V^{\pi^k}(s)\right]$, where N_k represents the number of rounds of interaction with the environment in this iteration. Furthermore, we can



(a) LAICA

(b) Cal-LAICA

Figure 4: The figures demonstrate the motivation of the proposed method. The y-axis of each figure denotes the Q-function of action representations on embedding space, and the x-axis denotes the 1-D action embedding space, where $e = \phi^{-1}(a)$. When the action space is changed, if there exist two actions a_1 and a_2 satisfy $\phi_{k+1}^{-1}(a_2) \subseteq \phi_k^{-1}(a_1)$, as illustrated in Figure 3. If the underestimation of action value functions for a_1 happens during the beginning of online policy improvement (right figure of (a)), LAICA needs a certain number of samples to correct the underestimation. On the other hand, Cal-LAICA draws ideas from offline-to-online methods and calibrates the values to avoid underestimation, thereby improving the performance during adaptation.

define the total regret obtained from adapting interactions over the entire life cycle of the model:

where K denotes the number of changes in the action space, and N_k represents the number of interaction rounds with the environment for each action space change. Through this definition, we compute the total regret over the entire lifecycle of the proposed method. Additionally, we further define the regret generated from learning in the latent action space as:

$$Reg(K) := \mathbb{E}_{s\sim\mathcal{D}} \sum_{k=1}^{K} \sum_{n=1}^{N_k} \left[V^{\beta_{\pi^\star}}(s) - V^{\beta_{\pi^{k,n}}}(s) \right] \quad (11)$$

The following theorem gives the bound of regret:

Theorem 1 *(Regret bound of Cal-LAICA): With high probability, Cal-LAICA obtains the following bound on the total regret accumulated during the adaptation:*

$$\begin{aligned} Reg(K) =& \tilde{\mathcal{O}}(\min\{C_{\beta_{\pi^\star}}^{\beta_\mu} H\sqrt{dN\log(|\mathcal{F}|)}, \\ & N\mathbb{E}_\rho\left[V^{\beta_{\pi^\star}}(s_0) - V^{\beta_\mu}(s_0) + H\sqrt{d_{\beta_\mu} N\log(|\mathcal{F}|)}\right]\}) \end{aligned} \quad (12)$$

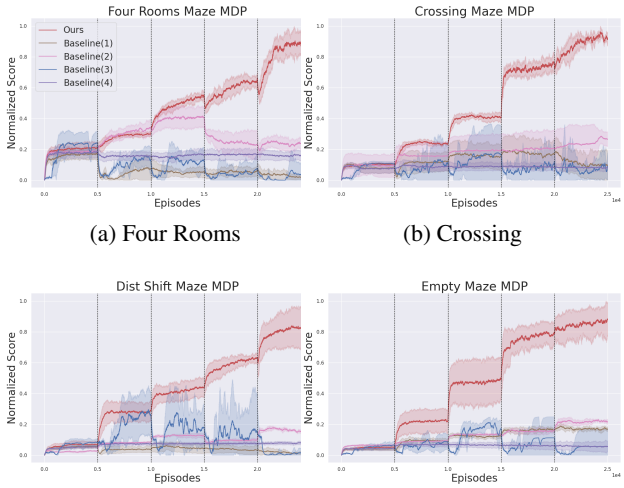
where $N = \sum_{k=1}^{K} N_k$, and $\mathcal{F}$ is the functional class of the Q-function.$C_{\beta_{\pi^\star}}^{\beta_\mu}$ is the Calibrated Bellman error transfer coefficient (Nakamoto et al. 2024).d is the bellman bilinear rank(Song et al. 2022).

The detailed proof is provided in the appendix.

Improving the Data Efficiency for Lifelong RL

Existing lifelong learning methods initialize a replay buffer for adapting when the action space changes (in Phase 1) (Chandak et al. 2020). When policy improvement begins, the above replay buffer is discarded, which leads to a waste of samples. In addition, in the policy improvement (Phase 2), the representation policy is trained using samples collected only with the same MDP $\mathcal{M}_k$, which is also inefficient in learning. Specifically, when the new action set is introduced to the environment for lifelong RL, the state action distribution shifts inevitably. This shift is similar to the offline-to-online RL works, as those methods utilize samples collected online to fine-tune the value functions learned from offline samples (Lee et al. 2022; Zheng et al. 2023; Song et al. 2022). Therefore, to fully use the samples generated in Phase 1, we use a simple but effective online-offline replay buffer (OORB) similar to (Zheng et al. 2023; Nakamoto et al. 2024) in Cal-LAICA to address the distribution shift. We store the transitions from the previous MDP M_{k-1} in the offline buffer, and the transitions closer to the current policy under the current MDP M_k are stored in the online buffer. The data from OORB are sampled under a Bernoulli distribution. With a probability p, transitions are sampled from the offline buffer. Otherwise, the transitions are sampled from the online buffer. p is regarded as a hyperparameter during learning.

Finally, we add the model pre-train process before the first change of the action space to further improve learning efficiency. The pseudo-code of Cal-LAICA is illustrated in the appendix.



(a) Four Rooms (b) Crossing

(c) Dist Shift (d) Empty Maze

Figure 5: Performance evaluation on the four tasks. The shaded regions represent the standard deviations of five independent runs on different random seeds. The vertical dashed lines denote the time when action space changed.

Experiments

In this section, we design experiments to validate the effectiveness of Cal-LAICA. We compare Cal-LAICA with the following methods:

- Baseline(1): it's a modified version of Cal-QL, an online-to-offline RL method. As the original Cal-QL cannot adapt to the changing action set, we reinitialized it when the action space changes. Each reinitialized model is trained offline prior to online improvement. The data is collected by running the SAC algorithm.
- Baseline(2): the policy is reinitialized after each change in the action space similar to Baseline(1), but uses high-quality expert data for offline training with CQL (Kumar et al. 2020).
- Baseline(3): The original LAICA (Chandak et al. 2020) algorithm. We use SAC to train the action representation policy.
- Baseline(4): A similar implementation to Baseline(1), but uses the offline-to-online method Cal-QL (Nakamoto et al. 2024) for training. The policy is not initialized after the changes in the action space.
- Cal-LAICA: our proposed method. Similarly to LAICA, we use CQL to pre-train the action representation policy.

To illustrate the effectiveness of Cal-LAICA, we evaluate the proposed methods and the above baseline methods on three tasks in the maze environment from MiniGrid: Four Rooms, Crossing and Dist Shift Maze. We measure the normalized total return for each task. The action space for those tasks is the same, with 8 actuators and a total of 2^8 discrete actions. For each task, we randomly divided the 2^8 actions into five sets and gradually added these sets to the available action space to simulate a dynamically changing action set. In addition, when the action space changes, new goals are also added to the environment. The agent's goal is to reach all the goals. The details of the environment can be found in the Appendix.

Emperical Results

Figure 5 demonstrates the performance comparison between Cal-LAICA and four baseline methods. In the four tasks, the proposed Cal-LAICA consistently outperforms the baseline methods. Cal-LAICA not only achieves better normalized total returns in each episode but also exhibits rapid adaptation capabilities when new actions are introduced into the action space. Specifically, compared to Baseline(1) and Baseline(2), Cal-LAICA not only shows superior performance but also incurs significantly lower data collection costs than Baseline(2). As the action space changes, providing the agent with high-quality and reliable experiences becomes increasingly challenging for reinforcement learning methods. The results also validate the findings in (Taiga et al. 2023): even after extensive training, lifelong reinforcement learning outperforms methods that learn from scratch. Compared to Baseline(3), we observe that Baseline(3) fails to learn quickly in most tasks. After the changes in the action space, inaccurate Q-value estimation leads to random updates of the policy, and such updates are often harmful to learning. As the number of available actions continues to increase, calibrating the estimation for action values becomes increasingly challenging. Compared to Baseline(4), our method retains the advantages of lifelong RL algorithms, enabling continuous learning without re-parameterization when facing dynamically changing action sets. Compared to existing offline-to-online reinforcement learning approaches, our algorithm effectively leverages the strengths of lifelong RL algorithms, significantly improving the generalization ability of current offline-to-online methods when dealing with changing action sets.

Result Analysis

To verify the effectiveness of calibrated Q-value estimation in lifelong learning when dealing with changing action sets, we compare the averaged Q-value of Cal-LAICA with the original LAICA during the learning process in the Four Rooms task. The results are shown in Fig. 6. We specifically focus on the adaptation process when new actions are added. As shown in Figures 6a and 6b, after the first change in the action space, the LAICA algorithm experiences a drop in the Q-value but eventually recovers, with its performance restored. However, after the second and third action space changes, the learned average Q-value fails to converge and almost completely loses the previously learned experiences. In contrast, Cal-LAICA helps the agent learn more smoothly through adapting to the new action spaces three times, enabling efficient learning and better task adaptation. In addition, as shown in Figures 6c and 6d, after the first two action space changes, the average Q-value learned by Cal-LAICA stops decreasing after approximately 200 and 400 episodes of training, respectively. LAICA requires about 1000 episodes of training to prevent the decline after the first action space change, and after the second action space change, the learned average Q-value fails to converge.

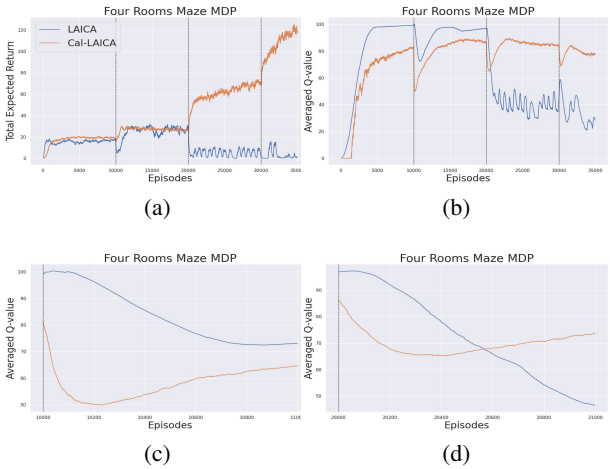


Figure 6: (a) The performance comparison between LAICA and Cal-LAICA which is measured by the total return per episode. (b) The averaged Q-values of each episode during the learning process of the two algorithms. (c) The average Q-values of the two algorithms after the first changes of the action space (episode 10000) (d) The average Q-values of the two algorithms after the second changes of the action space (episode 20000)

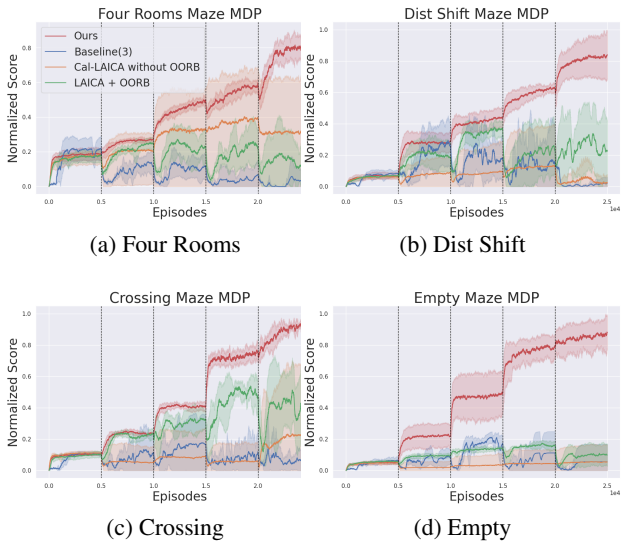


Figure 7: Ablation studies of the four tasks. The results are averaged on five runs with different random seeds. The vertical dashed lines denote the time when action space changed.

In contrast, Cal-LAICA can effectively constrain the lower bound of estimated Q-value during the adaptation to new action spaces by referencing calibrated policies, helping the Agent to transition more smoothly to new action spaces and tasks. Therefore, the proposed calibration method significantly alleviates the forgetting problem caused by erroneous estimates, demonstrating its effectiveness in lifelong RL.

Ablation Experiments

In this section, we conduct ablation studies to examine the independent contributions of the two components in our proposed Cal-LAICA method. As described in Section , our method first improves sample efficiency by using a simple yet effective Offline-Online ReplayBuffer (OORB). Additionally, we introduce a regularization tool to prevent the policy from making arbitrary updates, which could lead to performance degradation due to the underestimation of the Q-values.

In the experiments, we remove each component and compare the performance of the Cal-LAICA and LAICA algorithms across the four tasks in Section to assess their impact. The results show that Cal-LAICA is still able to learn quickly even after changes in the action space. Furthermore, LAICA+OORB, while performing worse than Cal-LAICA across the four tasks, still outperforms LAICA despite experiencing unlearning after changes in the action space. This indicates that the calibration tool is a key component for Cal-LAICA's ability to quickly learn under a new action set. Finally, in the experiment where OORB is removed, we observe that, except for the Four Rooms environment, where the agent's normalized score is still better than LAICA, the performance on other tasks is inferior to that of LAICA. This suggests that the calibration component almost completely loses its effectiveness, primarily because the reference policy used by the calibration tool is based on past behavior policies. Once these samples are discarded, the calibration component is unable to correct β's Q-estimation of the behavior representation policy.

These experimental results demonstrate that Cal-LAICA plays a crucial role in enabling lifelong RL to quickly adapt to new action sets. Its ability to rapidly and continuously learn in new tasks, along with its effective use of past samples, allows Cal-LAICA to outperform other methods across multiple tasks.

Conclusion

In this work, we develop Cal-LAICA, a method for calibrating value functions in lifelong reinforcement learning for changing action sets. Cal-LAICA adopts a conservative value estimation method to avoid underestimation, leading to an efficient adaptation when generalizing to new actions. We further analyzed the regret bound of the proposed method. In addition, we designed a replay buffer to store all the transitions collected for learning to further improve the sample efficiency. One drawback of Cal-LAICA is that it can only deal with discrete action spaces. In the future, we will study to improve Cal-LAICA towards lifelong learning tasks with continuous action spaces.

References

Afsar, M. M.; Crump, T.; and Far, B. 2022. Reinforcement learning based recommender systems: A survey. *ACM Computing Surveys*, 55(7): 1–38.

Akkerman, F.; Luy, J.; van Heeswijk, W.; and Schiffer, M. 2023. Dynamic Neighborhood Construction for Structured Large Discrete Action Spaces. *arXiv preprint arXiv:2305.19891*.

Ball, P. J.; Smith, L.; Kostrikov, I.; and Levine, S. 2023. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*, 1577–1594. PMLR.

Barreto, A.; Dabney, W.; Munos, R.; Hunt, J. J.; Schaul, T.; van Hasselt, H. P.; and Silver, D. 2017. Successor features for transfer in reinforcement learning. *Advances in neural information processing systems*, 30.

Bertoin, D.; and Rachelson, E. 2021. Local Feature Swapping for Generalization in Reinforcement Learning. In *International Conference on Learning Representations*.

Boopathy, A.; Liu, K.; Hwang, J.; Ge, S.; Mohammedsaleh, A.; and Fiete, I. R. 2023. Model-agnostic measure of generalization difficulty. In *International Conference on Machine Learning*, 2857–2884. PMLR.

Bunne, C.; Krause, A.; and Cuturi, M. 2022. Supervised training of conditional monge maps. *Advances in Neural Information Processing Systems*, 35: 6859–6872.

Chandak, Y.; Theocharous, G.; Kostas, J.; Jordan, S.; and Thomas, P. 2019. Learning Action Representations for Reinforcement Learning. In Chaudhuri, K.; and Salakhutdinov, R., eds., *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, 941–950. PMLR.

Chandak, Y.; Theocharous, G.; Nota, C.; and Thomas, P. 2020. Lifelong learning with a changing action set. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34(04), 3373–3380.

Fan, J.; and Li, W. 2022. Dribo: Robust deep reinforcement learning via multi-view information bottleneck. In *International Conference on Machine Learning*, 6074–6102. PMLR.

Farquhar, G.; Gustafson, L.; Lin, Z.; Whiteson, S.; Usunier, N.; and Synnaeve, G. 2020. Growing action spaces. In *International Conference on Machine Learning*, 3040–3051. PMLR.

Fujimoto, S.; Chang, W.-D.; Smith, E.; Gu, S. S.; Precup, D.; and Meger, D. 2024. For sale: State-action representation learning for deep reinforcement learning. *Advances in Neural Information Processing Systems*, 36.

Garcia, F.; and Thomas, P. S. 2019. A meta-MDP approach to exploration for lifelong reinforcement learning. *Advances in Neural Information Processing Systems*, 32.

Ghahramani, Z. 2001. An introduction to hidden Markov models and Bayesian networks. *International journal of pattern recognition and artificial intelligence*, 15(01): 9–42.

Ghosh, D.; Rahme, J.; Kumar, A.; Zhang, A.; Adams, R. P.; and Levine, S. 2021. Why generalization in rl is difficult: Epistemic pomdps and implicit partial observability. *Advances in neural information processing systems*, 34: 25502–25515.

Guo, S.; Zou, L.; Chen, H.; Qu, B.; Chi, H.; Philip, S. Y.; and Chang, Y. 2023. Sample efficient offline-to-online reinforcement learning. *IEEE Transactions on Knowledge and Data Engineering*.

Igl, M.; Ciosek, K.; Li, Y.; Tschiatschek, S.; Zhang, C.; Devlin, S.; and Hofmann, K. 2019. Generalization in reinforcement learning with selective noise injection and information bottleneck. *Advances in neural information processing systems*, 32.

Jain, A.; Kosaka, N.; Kim, K.-M.; and Lim, J. J. 2022a. Know your action set: Learning action relations for reinforcement learning. In *International Conference on Learning Representations*.

Jain, A.; Kosaka, N.; Kim, K.-M.; and Lim, J. J. 2022b. Know Your Action Set: Learning Action Relations for Reinforcement Learning. In *International Conference on Learning Representations*.

Jain, A.; Szot, A.; and Lim, J. 2020. Generalization to New Actions in Reinforcement Learning. In III, H. D.; and Singh, A., eds., *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, 4661–4672. PMLR.

Ju, H.; Juan, R.; Gomez, R.; Nakamura, K.; and Li, G. 2022. Transferring policy of deep reinforcement learning from simulation to reality for robotics. *Nature Machine Intelligence*, 4(12): 1077–1087.

Khetarpal, K.; Riemer, M.; Rish, I.; and Precup, D. 2022. Towards continual reinforcement learning: A review and perspectives. *Journal of Artificial Intelligence Research*, 75: 1401–1476.

Kirk, R.; Zhang, A.; Grefenstette, E.; and Rocktäschel, T. 2023. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76: 201–264.

Korkmaz, E. 2024. A Survey Analyzing Generalization in Deep Reinforcement Learning. *https://arxiv.org/pdf/2401.02349v2*.

Kostrikov, I.; Nair, A.; and Levine, S. 2021. Offline reinforcement learning with implicit q-learning. *arXiv preprint arXiv:2110.06169*.

Kumar, A.; Zhou, A.; Tucker, G.; and Levine, S. 2020. Conservative q-learning for offline reinforcement learning. *Advances in Neural Information Processing Systems*, 33: 1179–1191.

Lan, L.-C.; Zhang, H.; and Hsieh, C.-J. 2023. Can Agents Run Relay Race with Strangers? Generalization of RL to Out-of-Distribution Trajectories. In *International Conference on Learning Representation*. ICLR.

Lecarpentier, E.; Abel, D.; Asadi, K.; Jinnai, Y.; Rachelson, E.; and Littman, M. L. 2021. Lipschitz lifelong reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35(9), 8270–8278.

Lee, S.; Seo, Y.; Lee, K.; Abbeel, P.; and Shin, J. 2022. Offline-to-online reinforcement learning via balanced replay and pessimistic q-ensemble. In *Conference on Robot Learning*, 1702–1712. PMLR.

Levine, S.; Kumar, A.; Tucker, G.; and Fu, J. 2020. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*.

Li, W.; Wang, X.; Jin, B.; Luo, D.; and Zha, H. 2021. Structured cooperative reinforcement learning with time-varying composite action space. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(11): 8618–8634.

Li, W.; Wang, X.; Jin, B.; Luo, D.; and Zha, H. 2022. Structured Cooperative Reinforcement Learning With Time-Varying Composite Action Space. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(11): 8618–8634.

Lyle, C.; Rowland, M.; Dabney, W.; Kwiatkowska, M.; and Gal, Y. 2022. Learning dynamics and generalization in reinforcement learning. *arXiv preprint arXiv:2206.02126*.

Matsushima, T.; Furuta, H.; Matsuo, Y.; Nachum, O.; and Gu, S. 2020. Deployment-efficient reinforcement learning via model-based offline optimization. *arXiv preprint arXiv:2006.03647*.

Mazoure, B.; Ahmed, A. M.; MacAlpine, P.; Hjelm, R. D.; and Kolobov, A. 2022. Cross-Trajectory Representation learning for zero-shot generalization in RL. *International Conference on Learning Representations*.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-level control through deep reinforcement learning. *nature*, 518(7540): 529–533.

Nair, A.; Gupta, A.; Dalal, M.; and Levine, S. 2020. Awac: Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*.

Nakamoto, M.; Zhai, S.; Singh, A.; Sobol Mark, M.; Ma, Y.; Finn, C.; Kumar, A.; and Levine, S. 2024. Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning. *Advances in Neural Information Processing Systems*, 36.

Ni, T.; Eysenbach, B.; and Salakhutdinov, R. 2022. Recurrent Model-Free RL Can Be a Strong Baseline for Many POMDPs. In *International Conference on Machine Learning*, 16691–16723. PMLR.

Pong, V. H.; Nair, A. V.; Smith, L. M.; Huang, C.; and Levine, S. 2022. Offline meta-reinforcement learning with online self-supervision. In *International Conference on Machine Learning*, 17811–17829. PMLR.

Raileanu, R.; Goldstein, M.; Szlam, A.; and Fergus, R. 2020. Fast adaptation to new environments via policy-dynamics value functions. In *Proceedings of the 37th International Conference on Machine Learning*, 7920–7931.

Segler, M. H.; Preuss, M.; and Waller, M. P. 2018. Planning chemical syntheses with deep neural networks and symbolic AI. *Nature*, 555(7698): 604–610.

Silver, D.; Huang, A.; Maddison, C. J.; Guez, A.; Sifre, L.; Van Den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; et al. 2016. Mastering the game of Go with deep neural networks and tree search. *nature*, 529(7587): 484–489.

Sinii, V.; Nikulin, A.; Kurenkov, V.; Zisman, I.; and Kolesnikov, S. 2023. In-Context Reinforcement Learning for Variable Action Spaces. *arXiv preprint arXiv:2312.13327*.

Song, Y.; Zhou, Y.; Sekhari, A.; Bagnell, J. A.; Krishnamurthy, A.; and Sun, W. 2022. Hybrid rl: Using both offline and online data can make rl efficient. *arXiv preprint arXiv:2210.06718*.

Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement learning: An introduction*. MIT press.

Taiga, A. A.; Agarwal, R.; Farebrother, J.; Courville, A.; and Bellemare, M. G. 2023. Investigating multi-task pretraining and generalization in reinforcement learning. In *The Eleventh International Conference on Learning Representations*.

Vinyals, O.; Babuschkin, I.; Czarnecki, W. M.; Mathieu, M.; Dudzik, A.; Chung, J.; Choi, D. H.; Powell, R.; Ewalds, T.; Georgiev, P.; et al. 2019. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *nature*, 575(7782): 350–354.

Wang, H.; Zheng, S.; Xiong, C.; and Socher, R. 2019. On the generalization gap in reparameterizable reinforcement learning. In *International Conference on Machine Learning*, 6648–6658. PMLR.

Yasutomi, A. Y.; and Ogata, T. 2023. Automatic Action Space Curriculum Learning with Dynamic Per-Step Masking. In *2023 IEEE 19th International Conference on Automation Science and Engineering (CASE)*, 1–7. IEEE.

Ye, J.; Li, X.; Wu, P.; and Wang, F. 2023. Action Pick-up in Dynamic Action Space Reinforcement Learning. *arXiv preprint arXiv:2304.00873*.

Yu, Z.; and Zhang, X. 2023. Actor-critic alignment for offline-to-online reinforcement learning. In *International Conference on Machine Learning*, 40452–40474. PMLR.

Zheng, H.; Luo, X.; Wei, P.; Song, X.; Li, D.; and Jiang, J. 2023. Adaptive policy learning for offline-to-online reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37(9), 11372–11380.

A. Related Works

Reinforcement Learning for Unseen Action Spaces

Offline-to-Online Reinforcement Learning

Online RL methods always need numerous of samples to learn from scratch (Levine et al. 2020). Using pre-collected offline data can accelerate the learning of online RL methods, which lead to offline-to-online RL methods (Nair et al. 2020; Matsushima et al. 2020). For offline-to-online RL, there are two common approaches. The first approach directly utilize offline data as online samples, which improves sample efficiency but does not avoid the issue that the policies executed early in training are typically poor policies (Song et al. 2022). The second approach first performs offline reinforcement learning to obtain a good initial policy to avoid the challenges associated with executing poor policies early in online training and then transitions to normal online training (Nakamoto et al. 2024; Zheng et al. 2023). A common challenge faced by offline-to-online RL work is the issue of distributional shift, where the agent encounters previously unseen state-action pairs (Lee et al. 2022; Song et al. 2022). This is similar to the problem in lifelong RL research. Studies like (Nair et al. 2020; Pong et al. 2022; Ball et al. 2023; Guo et al. 2023) have explored the design of hybrid offline-online replay buffers to address this challenge. In addition, to address the issue of overly optimistic action-value estimates in the online state, methods such as (Nakamoto et al. 2024; Yu and Zhang 2023) primarily smooth the Q-values of offline policy actions and non-offline actions for rapid learning. Approaches like (Kostrikov, Nair, and Levine 2021; Nair et al. 2020; Lee et al. 2022) tend to favor actions closer to the offline policy to handle the distributional shift challenge. (Zheng et al. 2023) applies an adaptive updating scheme, using a pessimistic update strategy for offline datasets and an optimistic/greedy update strategy for online datasets.

Reinforcement Learning for Unseen Action Spaces

Studies such as (Ghahramani 2001; Bunne, Krause, and Cuturi 2022; Fujimoto et al. 2024) have explored a novel framework in which the agent learns a latent representation of actions. This approach groups semantically similar actions into a latent action space by capturing the semantic information of the actions, This approach helps avoid the limitations imposed by the scale of state and action spaces. Research on variable action spaces also builds on these concepts. In particular, (Chandak et al. 2019) describes this semantic relationship as the underlying structure of actions. The description of semantic information between actions in this paper primarily draws on this concept.

Conventional reinforcement learning algorithms often struggle with handling variable action sets. To address this, some researchers focus on effectively reusing model parameters to manage the addition of new actions to the action set (Li et al. 2021; Sinii et al. 2023). Other studies, such as (Jain, Szot, and Lim 2020; Jain et al. 2022b; Farquhar et al. 2020; Ye et al. 2023), primarily explore embedding actions into a latent space to evaluate unknown actions, enabling the agent to adapt to unseen actions.

However, these approaches often lack the ability to continuously optimize, which poses significant challenges in tasks like robot control. For example, if a robot initially only has a ”move forward” action, the evaluation of a newly added ”move backward” action would be highly unreliable. To address this limitation, another line of research focuses on continual learning. For instance, (Chandak et al. 2020) extends the work of (Chandak et al. 2019) by decomposing the policy learning task into two components: (1) learning the relationship between actions and the action representation space, and (2) reasoning and learning a representation policy within that action representation space. This decomposition facilitates continual learning.

Additionally, studies such as (Akkerman et al. 2023; Yasutomi and Ogata 2023)though not strictly focused on variable action spacesaim to investigate large discrete action sets by reducing learning difficulty through the gradual involvement of new actions.

Lifelong Reinforcement Learning

In lifelong RL, the agent aims to learn a generalized policy through different tasks. This is an important research topic in the field of online RL (Kirk et al. 2023; Khetarpal et al. 2022; Korkmaz 2024; Barreto et al. 2017; Lecarpentier et al. 2021; Garcia and Thomas 2019; Fan and Li 2022; Ni, Eysenbach, and Salakhutdinov 2022). It is important to note that the generalization in RL differs fundamentally from that in supervised learning. In general, an RL agent designed to master tasks should be able to achieve its goals from any controllable state in the environment, rather than simply memorizing a small set of trajectories (Lan, Zhang, and Hsieh 2023). Research such as (Wang et al. 2019; Ghosh et al. 2021; Boopathy et al. 2023) explores the definition of generalization in RL and proposes new metrics for measuring the difficulty of generalization. Studies like (Igl et al. 2019; Bertoin and Rachelson 2021) mainly focus on designing regularization methods and adjusting those commonly used in supervised learning to suit the structure of RL, thereby improving the models generalization across different tasks and preventing overfitting to a single task. (Lyle et al. 2022) proposes a post-training policy distillation method to enhance the policys generalization across various tasks. (Raileanu et al. 2020).

B. Implementation Details of Cal-LAICA

A Python-style pseudocode of the proposed Cal-LAICA algorithm is illustrated in Table 1. Additionally, a detailed implementation of Cal-LAICA can be found in anonymous Github: https://anonymous.4open.science/r/Cal-LAICA-E68A/

Table 1: Cal-LAICA python Pseudocode

```
# Input: Action representation function beta, action mapping function phi, inverse dynamic function psi
Initialize beta, phi, psi

offline_pretrain() # Offline pre-train the models based on C-QL
for change in range(change_count):
    Update the available action set;
    if not online_buffer.empty():
        Add online_buffer to offline_buffer; # adding the buffer collected previously to the offline buffer
    online_buffer.clear()
    explore(ReplayBuffer = online_buffer) # Explore the environment
    for iter in (iterations):
        batch = online_buffer.sample()
        update phi and psi by maximizing(phi,psi) #Updata \phi and \psi
        pass
    while not change_action:
        for t in range(0,1,2,...):
            if random.random()<p:
                batch=offline_buffer.sample()
                pass
            else:
                batch=online_buffer.sample()
                pass
            e=beta.get_action(state) # Obtain action embeddings under a specific state
            a=phi(e) # Mapping the action embedding to original actions
            update(beta,batch) # Update beta function with sampled batch
            update Critic by Minimizizing TD-Error # Using the regularizer in Q-function
    until convergence
```

The proposed work is an improved version of the LAICA algorithm, which improves the learning efficiency in adapting to changing action spaces. We utilize the offline-to-online learning method in lifelong RL with changing action sets, to calibrate the value estimation in lifelong reinforcement learning. The difference between Cal-LAICA, LAICA and offline-to-online methods is depicted in 8.

C. Experiment Details

C.1 Maze Details

We conduct experiments on three maze tasks from MiniGrid[2]. Figure 9 shows the details of the environments. For each tasks, the agent controls a robot with eight actuators in different directions. There are a total 2^8 actions in the whole action space. To simulate the lifelong learning tasks with changing action spaces, the agent only knows part of the actuators and the actuators will continually activate during the learning process. The agent's ultimate goal is to reach a goal position (green in Figure 8) unless the maximum step is reached. To simplify the exploration process and simulate the lifelong RL process, we add a series of subgoals that lead the agent to the final goal. When a subgoal is reached, the agent receives a positive reward signal. The details of the environments are listed below:

State: The state space contains the position of the agent, and the position of the current goal. Therefore, the observation of each step is a 4-D real vector. We linearly normalized the coordinates for each dimension into $[0, 1]$.

Actions: As illustrated before, there are total 2^8 actions in the whole action space. We divided the action set into five roughly equal parts, with the first four parts each containing 51 discrete actions and the last part containing 52 actions. Whenever a signal indicating a change in the action space is received, a new set of actions will be added to the available action set.

Reward: When the agent reaches a goal point, the reward is given a one-time reward, which is calculated as 100 divided by the steps used to reach the goal.

[2]https://minigrid.farama.org/index.html

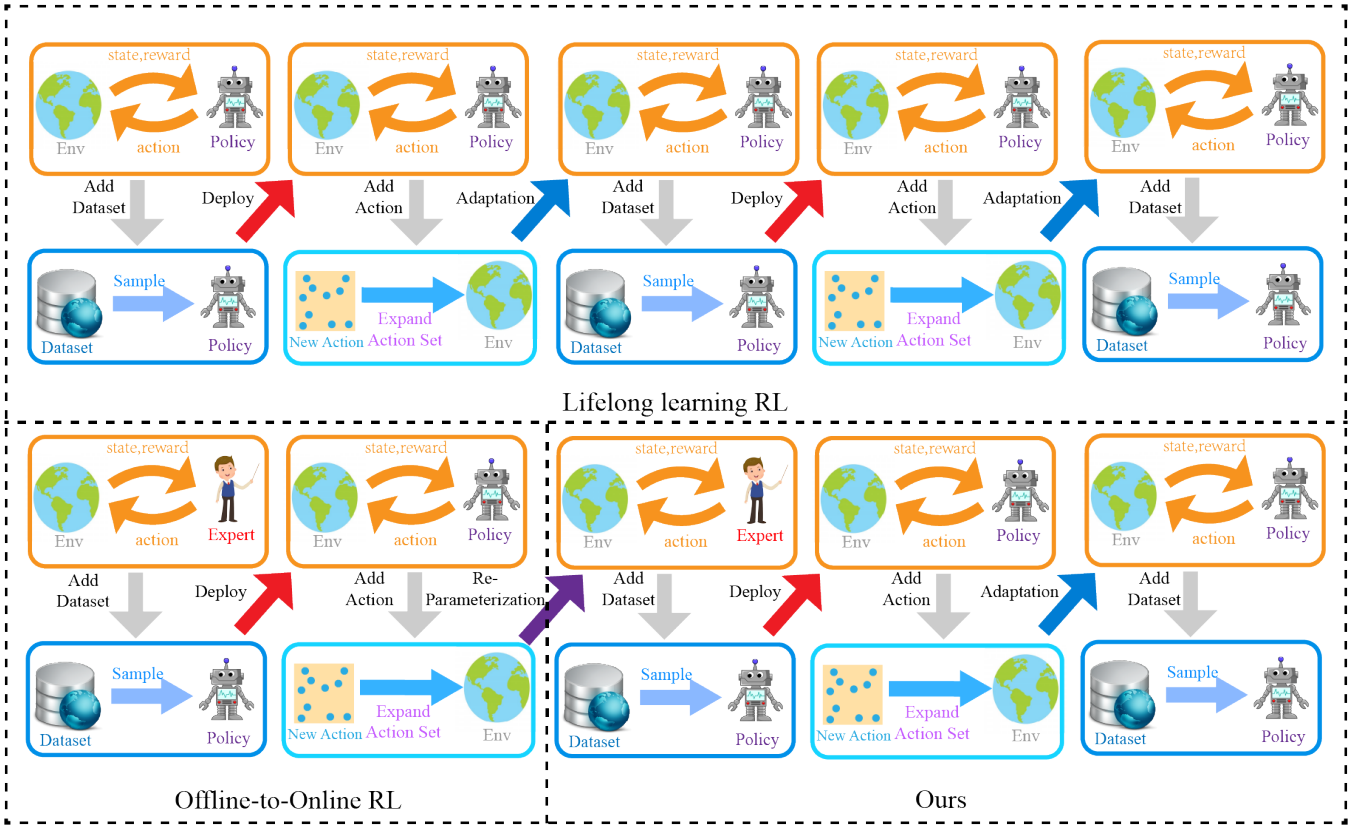


Figure 8: This figure demonstrates the difference between LAICA, the conventional offline-to-online methods, and the proposed method. LAICA is an online RL approach that can generalize RL to new action spaces without the requirement of demonstrations from experts. In contrast, conventional offline-to-online methods require the collection of data for pre-training and re-parameterizing when the action space changes. The proposed method only requires a certain amount of expert data for pre-training at the beginning of the learning but inherits LAICA's advantage in generalizing to changing action spaces.

C.2 Implementation Details

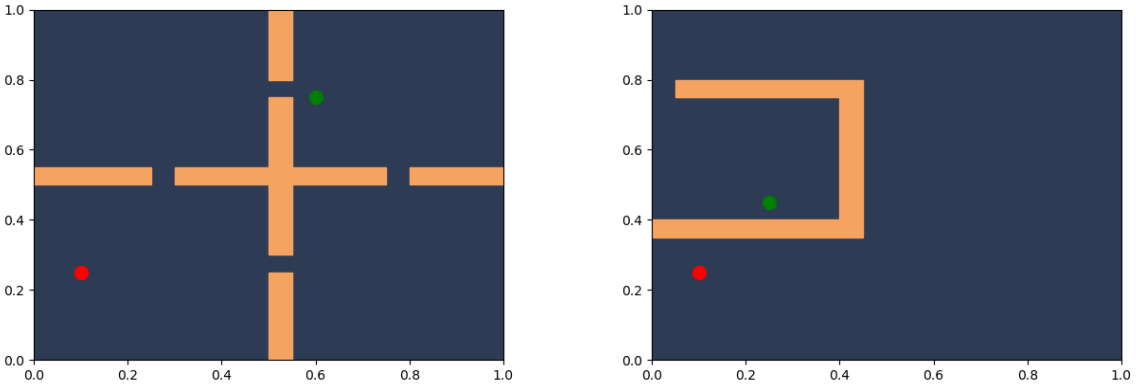
Baseline(1) Implemented based on the algorithm (Nakamoto et al. 2024). After each adjustment to the action space, the model is reinitialized to ensure that the algorithm can function correctly. To provide the offline dataset needed for model pre-training, we first train an online model for 1,000 episodes, which is then used to collect the offline dataset. After obtaining the offline dataset, the model undergoes 5,000 episodes of pre-training, followed by transitioning to online interaction to update the buffer. Whenever a new action set is added to the available set, the model is re-parameterized, and the process is repeated.

Baseline(2) Implemented based on the algorithm (Kumar et al. 2020). After each adjustment to the action space, the model is reinitialized. To provide the offline dataset needed for model training, we continue to use a dataset generated by experts, ensuring that the dataset is of sufficiently high quality, meaning it is entirely generated by experts.

Baseline(3) Implemented based on (Chandak et al. 2020), this model is capable of adapting to different action sets without requiring expert data throughout the process. The training data is entirely collected online by the model itself.

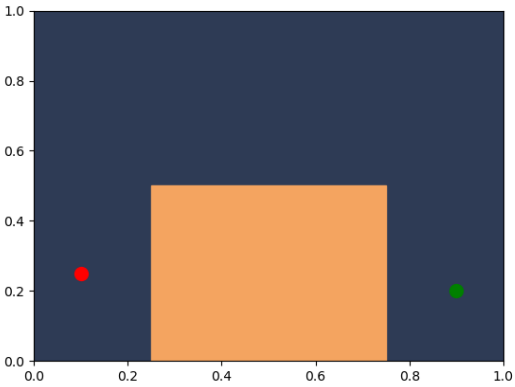
Baseline(4) Implemented based on (Nakamoto et al. 2024). After adjustments to the action space, the model is not reinitialized, nor is the available action set adjusted. The offline pre-training data is generated by experts, while subsequent online data is generated by the model itself. This model will undergo 5,000 episodes of pre-training in offline mode before transitioning to the online interaction stage.

Ours Our algorithm uses a high-quality dataset provided by an expert model only during the first action space adjustment (i.e., at the start of training), performing approximately 5,000 episodes of offline pre-training. The model then transitions to the normal online interaction stage.

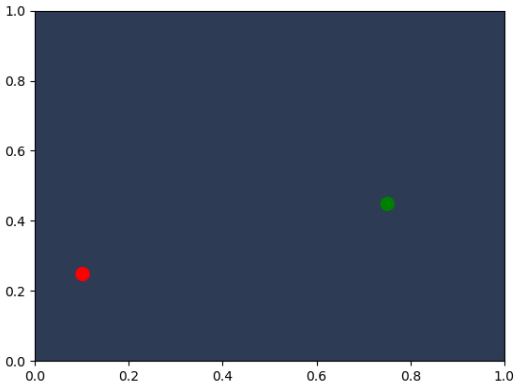


(a) Four Rooms Maze

(b) Crossing Maze



(c) Dist Shift Maze



(d) Empty Maze

Figure 9: Maze Domain

D Regret Analysis of Cal-LAICA

D.1 Definitions

Definition 1 *(Chandak et al. 2019).The state value function for strategies in spatial reasoning can be modified as:*

$$V^{\pi} = \sum_{a \in \mathcal{A}} \int_{\phi_{\pi}^{-1}(a)} \beta_{\pi}\left(e|s\right) Q^{\pi}\left(s,a\right) \mathrm{d}e \tag{13}$$

Definition 2 *(Song et al. 2022; Nakamoto et al. 2024).The definition of the symbols introduces the Bellman operator, resulting in the Bellman operator on the discrete action space:*

$$\mathcal{T}Q\left(s,a\right) = R\left(s,a\right) + \gamma \sum_{s'} P\left(s'|s,a\right) V^{\pi}\left(s\right) \tag{14}$$

Definition 3 *(Song et al. 2022).For any policy π that performs inference learning in the action space, we define the Bellman error transfer coefficient as:*

$$C_{\pi} := \max\left\{0, \max_{f \in \mathcal{F}} \frac{\sum_{h=0}^{H-1} \mathbb{E}_{s,a \sim d_h^{\pi}}\left[\mathcal{T}f_{h+1}(s,a) - f_h(s,a)\right]}{\sqrt{\sum_{h=0}^{H-1} \mathbb{E}_{s,a \sim \nu_h}\left(\mathcal{T}f_{h+1}(s,a) - f_h(s,a)\right)^2}}\right\} \tag{15}$$

where $\mathcal{F}$ is the functional class of the Q-function.

Definition 4 *For any policy , we define the Calibrated Bellman Error Transfer Coefficient with respect to the reference policy* π_{ref}

$$C_{\pi}^{\mathrm{ref}} := \max_{f \in \mathcal{F}, f(s,a) \geq Q^{\mathrm{ref}}(s,a)} \frac{\sum_{h=0}^{H-1} \mathbb{E}_{s,a \sim d_h^{\pi}} \left[\mathcal{T} f_{h+1}(s,a) - f_h(s,a) \right]}{\sqrt{\sum_{h=0}^{H-1} \mathbb{E}_{s,a \sim \nu_h} \left(\mathcal{T} f_{h+1}(s,a) - f_h(s,a) \right)^2}} \tag{16}$$

Our Result

We consider the definition of the Q function.

$$\begin{aligned} Q(s,a) &= \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t | s_0 = s, a_0 = a) \right] \\ &= \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \phi(e_t) | s_0 = s, \phi(e_0) = a) \right] \end{aligned} \tag{17}$$

Note that ϕ is an injection, so we have

$$Q(s,a) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, e_t | s_0 = s, e_0 = e) \right] := Q^{\beta_\pi}(s,e) \tag{18}$$

Then we can note that, according to the characteristic function setting,

$$\begin{aligned} V^{\pi}(s) &= \sum_{a \in \mathcal{A}} \int_{\phi_{\pi}^{-1}(a)} \beta_{\pi}(e|s) Q^{\pi}(s,a) \, \mathrm{d}e \\ &= \sum_{a \in \mathcal{A}} \int_{\phi_{\pi}^{-1}(a)} \beta_{\pi}(e|s) Q^{\pi}(s,e) \, \mathrm{d}e := V^{\beta_\pi}(s) \end{aligned} \tag{19}$$

Definition 5 *(Representation Bellman Error Transmission Coefficient).Thus, we can modify the action representation space's Bellman Error Transfer Coefficient and obtain the following definition:*

$$C_{\beta_\pi} := \max \left\{ 0, \max_{f^{\beta_\pi} \in \mathcal{F}} \frac{\sum_{h=0}^{H-1} \mathbb{E}_{s,e \sim d_h^{\pi}} \left[\mathcal{T} f_{h+1}^{\beta_\pi}(s,e) - f_h^{\beta_\pi}(s,e) \right]}{\sqrt{\sum_{h=0}^{H-1} \mathbb{E}_{s,e \sim \nu_h} \left(\mathcal{T} f_{h+1}^{\beta_\pi}(s,e) - f_h^{\beta_\pi}(s,e) \right)^2}} \right\} \tag{20}$$

C_{β_π} is mainly used to measure the performance and bias of the representation policy β_π on a given dataset. A larger C_{β_π} indicates a greater discrepancy between β_π and the dataset (Song et al. 2022).

Definition 6 *(Calibrated Representation Bellman Error Transmission Coefficient). According to 18, and in combination with the work in (Nakamoto et al. 2024), we can modify and obtain the definition of the Calibrated Representation Bellman Error Transmission Coefficient.*

$$C_{\beta_\pi}^{\mathrm{ref}} := \max_{f^{\beta_\pi} \in \mathcal{F}, f^{\beta_\pi}(s,e) \geq Q^{\mathrm{ref}}(s,e)} \frac{\sum_{h=0}^{H-1} \mathbb{E}_{s,e \sim d_h^{\beta_\pi}} \left[\mathcal{T} f_{h+1}^{\beta_\pi}(s,e) - f_h^{\beta_\pi}(s,e) \right]}{\sqrt{\sum_{h=0}^{H-1} \mathbb{E}_{s,e \sim \nu_h} \left(\mathcal{T} f_{h+1}^{\beta_\pi}(s,e) - f_h^{\beta_\pi}(s,e) \right)^2}} \tag{21}$$

Theorem 2 *(Regret Bound Analysis)By combining the theories from (Nakamoto et al. 2024) and (Chandak et al. 2019), we will operate in a finite-horizon setting with a horizon H.we can obtain an upper bound on the cumulative suboptimality characterizing a policy.:*

$$Reg(K) = \tilde{\mathcal{O}}(\min\{C_{\beta_{\pi^\star}}^{\beta_\mu} H \sqrt{dN \log(|\mathcal{F}|)}, N\mathbb{E}_{\rho}\left[V^{\beta_{\pi^\star}}(s_0) - V^{\beta_\mu}(s_0) + H\sqrt{d_{\beta_\mu} N \log(|\mathcal{F}|)} \right]\}) \tag{22}$$

The term $\mathcal{F}$ represents the class of Q-functions, d is the Bellman bilinear rank, and $N = \sum_{k=1}^{K_1} N_k$ is the total number of interaction steps between the model and the environment.

Proof.

$$
\begin{aligned}
Reg\left(K\right) &:= \sum_{k=1}^{K}\sum_{n=1}^{N_k} V^{\pi^\star} - V^{\pi^{k,n}} \\
&= \mathbb{E}_{s\sim\mathcal{D}} \sum_{k=1}^{K}\sum_{n=1}^{N_k} \left[V_0^{\pi^\star}(s) - V_0^{\pi^{k,n}}(s)\right] \\
&\mathbb{E}_{s\sim\mathcal{D}} \sum_{k=1}^{K}\sum_{n=1}^{N_k} \left[V_0^{\beta_{\pi^\star}}(s) - V_0^{\beta_{\pi^{k,n}}}(s)\right] \\
&\overset{(i)}{\leq} K_2\left(V^{\beta_{\pi^\star}}(s) - V^{\text{ref}}(s)\right) + K_1 C^{\text{ref}}_{\beta_{\pi^\star}}\sqrt{H\cdot\Delta_{\text{off}}} \\
&+ H\left(\sqrt{2d\log\left(1+\frac{K_1}{d}\right)\cdot(\Delta_{\text{on}}+B_X^2B_W^2)\cdot K_1} + \sqrt{2d_{\text{ref}}\log\left(1+\frac{K_2}{d_{\text{ref}}}\right)\cdot(\Delta_{\text{on}}+B_X^2B_W^2)\cdot K_2}\right) \\
&\overset{(ii)}{\leq} \tilde{\mathcal{O}}(\min\{C^{\beta_\mu}_{\beta_{\pi^\star}} H\sqrt{dN\log(|\mathcal{F}|)}, N\mathbb{E}_\rho\left[V^{\beta_{\pi^\star}}(s_0) - V^{\beta_\mu}(s_0) + H\sqrt{d_{\beta_\mu}N\log(|\mathcal{F}|)}\right]\})
\end{aligned}
\tag{23}
$$

Here, δ_{on} and δ_{off} follow the definitions in (Song et al. 2022),

$$
K_1 = \sum_{k=1}^{K}\sum_{n=1}^{N_k} \mathbb{1}\{f^{\beta_{\pi^{k,n}}}(s,e) > Q^{ref}(s,e)\}, \tag{24}
$$

$$
K_2 = \sum_{k=1}^{K}\sum_{n=1}^{N_k} \mathbb{1}\{f^{\beta_{\pi^{k,n}}}(s,e) \leq Q^{ref}(s,e)\}, \tag{25}
$$

and B_X and B_W follow the definitions in (Nakamoto et al. 2024). The derivations for (i) and (ii) can be directly obtained from (Nakamoto et al. 2024).